Ancestor Vectors

Motivation

`stockfoil` needs to support queries over the ancestors of various parts of a commit graph. A general use case is diffing between different states of a document, represented by a combination of two things:

- The delivered heads of the commit graph
- The write permissions, including per-author write frontiers

The delivered heads identify a causally complete history: the heads themselves and all their ancestors. Write permissions map author IDs (Ed25519 public keys) to an unbounded permission, a bounded frontier of commit hashes, or a denial. Missing authors are denied. A bounded frontier defines the down-set accepted for that author, subject to the frontier heads being in the delivered history. Delivery and visibility are different: an invisible commit can still have visible ancestors or descendants. Encountering it is not a reason to stop traversing the graph.

This makes visibility diffs more complex than ancestry differences alone. Permissions can change while delivered heads remain identical, changing the visibility of old commits. There is no general traversal cut at the intersection of the before/after write-frontier down-sets: permissions are per author, and visible commits need not form a down-set. Each candidate commit must instead be classified by its delivery and author permission in each view. The current diff iterator scans the stored topological prefix through the latest node in either delivery history, yielding commits whose visibility changes. It does not implement a frontier-to-head traversal.

In other scenarios materialization is split into two steps: first ingesting newly delivered, causally ready commits, then filtering commits by their visibility before and after some viewpoint. For example, Automerge stores all operations ever received in a log and materializes a view by walking that log and filtering out things which are not visible.

This second requirement motivates a simple way to check whether an individual commit is visible in a viewpoint, without walking its ancestry on every query.

Chain Clocks

Parent links already allow ancestry checks by graph traversal, but repeated walks are expensive in large graphs. The implementation in `src/ancestor_clock.rs` and `src/commit_graph.rs` instead uses graph-local chain clocks, with prepared visibility predicates in `src/view_point.rs`.

A chain is a set of commits totally ordered by ancestry. Each commit has an immutable chain ID and a position: its `NodeIdx` in the append-only, parents-before-children node store. Positions increase along a chain but need not be consecutive, and adjacent chain members need not share a direct parent edge. `NodeIdx(0)` is a real position, not an empty-clock sentinel.

A down-set intersects each chain in a prefix. `AncestorClock` represents it exactly as a sparse vector of `(chain ID, greatest included NodeIdx)` pairs, sorted by chain ID. An absent coordinate means that no member of that chain is included. A commit belongs to the down-set exactly when its chain coordinate is present and at least its position. This is a single coordinate lookup, not a comparison of whole vectors.

The clock at a commit includes that commit and all its ancestors. A clock for multiple heads is the union of their inclusive clocks, taking the maximum position for each chain. `ClockBuilder` batches these unions in a `BTreeMap` and emits a sorted vector. These are exact ancestry summaries, not estimates. For strict ancestry, `is_ancestor` first rejects equal or later node indices; an earlier

member of the same chain is immediately known to be an ancestor, otherwise it tests membership in the descendant's inclusive clock.

Chain Decomposition

Chains are assigned greedily as commits arrive with all parents present. The node store tracks the current tip of every chain. For a new commit:

- Prefer a direct parent which is still the current tip of its chain. If there are several eligible parents, choose the smallest chain ID.
- If no direct parent is eligible, form the union of the parents' ancestry clocks. Look for any currently tracked chain tip in this parent down-set, again choosing the smallest eligible chain ID. This tip can be an indirect ancestor of the new commit.
- Reuse that chain and advance its tip to the new commit's `NodeIdx`. If no chain is reusable, allocate a new chain ID with this node as its tip.

A root therefore allocates a new chain. Ordinary sequential edits use the direct-parent fast path without building clocks or scanning all chains. Reusing an indirect ancestor tip is important: direct-parent-only reuse would strand a chain after each diamond. Repeated fork-and-merge diamonds stay on two chains with this policy, rather than allocating a chain per diamond.

The invariant is that the previous tip is an ancestor of each new member. Existing labels never change. This is an online greedy decomposition, not a minimum-chain decomposition: it is not guaranteed to achieve the graph's optimal width, and the number of chains can depend on insertion order.

Computing and Caching Clocks

To compute a requested node's clock, walk backwards through parent links. Maintain a visited-node set and a `ClockBuilder`. At an uncached node, include its chain position and continue to its parents. At a node with a cached full ancestry clock, merge that clock and stop walking that branch: all of its ancestors are already represented.

It is not safe to stop just because the builder already contains a node's chain position. Its cross-chain parents may not yet have been visited. The safe shortcuts are revisits and cached **full ancestry** clocks, not partially accumulated coordinates. There is no nearest-containing-fragment lookup or containing-fragment index in this algorithm.

Each node has a `OnceLock<Arc<AncestorClock>>`. Computing a requested clock initializes that node's cache; the backward walk does not populate every intermediate node's cache. In addition, insertion eagerly requests a clock when any of these conditions holds:

- The commit's sedimentree hash depth is greater than zero.
- The commit has more than one parent entry.
- The longest tracked path gap since an eager checkpoint reaches 128 nodes.

The path-gap counter is one plus the maximum parent counter, or one at a root, and resets to zero at an eager checkpoint. Lazy query caching can shorten a later walk but does not reset this conservative counter. Hash-depth checkpoints are cached regardless of fragment certification; an ancestry clock is not a fragment certificate.

For a singleton set of heads, `clock_for_heads` shares the head's cached `Arc`. For multiple heads it merges their cached clocks into a new clock; an empty set produces an empty clock. When the requested delivery heads are exactly the current graph heads (after deduplication), the whole history contains every stored node. Its clock is constructed directly from the chain-tip table in linear time in the number of chains, with no ancestry walk.

Cached clocks are immutable: appending descendants cannot change a node's ancestry. Cloning a graph preserves the earlier prefix's indices and chain labels, and already initialized cache entries share their immutable clocks through `Arc`. Uninitialized entries can be filled independently after cloning. This does not make clocks interchangeable between independently built replicas, or between divergent suffixes of graph clones.

Normalized Fragment Membership

A fragment carries the head's inclusive history minus the union of all boundary histories. This difference is not itself a down-set: test inclusion in the head clock and exclusion from the boundary clock. Boundary discovery still uses the raw depth-stopped parent walk and retains redundant ancestor boundaries.

Each chain also stores its nodes in increasing position order. For each head clock coordinate, binary searches select the interval above the boundary clock's coordinate and at or below the head's coordinate. Enumerating these intervals visits normalized members without scanning the excluded history or unrelated nodes. Traversal, summaries, export, and local certificate construction use the same member calculation.

Prepared Viewpoints

The portable `ViewPoint` is unchanged: it owns delivered commit hashes and write permissions. It contains no chain labels, node indices, or ancestry clocks. Those are local implementation details, and are never transported or compared across replicas. A received viewpoint is resolved against admitted history; optimistic local diffing may also include pending local commits.

Preparing a viewpoint produces a `PreparedView` with:

- One inclusive delivery-history clock
- A set of authors with unbounded permission
- One bounded ancestry clock per author with a bounded permission

For each bounded frontier, first retain only frontier heads which are known and belong to this viewpoint's delivery history. Then union those retained heads' inclusive ancestry clocks. Filtering the heads **before** union is essential: an out-of-history bound grants nothing in this snapshot, even if some of its ancestors are delivered. Unioning all bounds and intersecting the result with delivery history would incorrectly grant those ancestors.

A commit is visible exactly when its position is in the delivery clock and its author is either unbounded or has a bounded clock containing that position. Denied or absent authors contribute nothing. Each membership query reads only the commit's own chain coordinate from each relevant clock: the delivery clock and, if needed, that author's bounded clock. No clock for the queried commit needs to be constructed. Diff classification applies this predicate to the before and after prepared views.

Delivery and individual bound clocks are down-sets. Visibility is not necessarily a down-set, because permissions filter commits by author. It therefore cannot in general be represented by a single visibility clock.

Costs and Limits

Let `K` be the number of coordinates in a relevant clock. Once prepared, membership uses binary search in the sparse sorted vector, taking `O(log K)` time (constant time for an empty clock). Visibility performs at most two such searches per view, in addition to hash-table lookups. Same-chain strict ancestry has a constant-time fast path; other ancestry queries may first need to compute the descendant's clock.

Preparation and initial computation are not free. A backward walk visits uncached nodes and edges and merges cached vectors. Every merge reads all coordinates of its input, and the current `BTreeMap` builder takes logarithmic time per coordinate update in the size of the accumulated clock. Multiple heads, bounded authors, and fallback chain assignment can require these unions. Checkpoint spacing limits uncached path lengths, not the total branching work or the size of the clocks being merged. Enumerating changed commits still scans a topological prefix; fast membership is not an output-sensitive diff.

Each cached clock uses space proportional to its own coordinate count, in addition to per-node metadata and the chain-tip table. Small chain counts make long, mostly sequential histories cheap, and `Arc` avoids copying shared immutable clocks. Wide histories or a poor greedy decomposition can make clocks large, however. With many large cached clocks, storage and accumulated construction work can still be quadratic in the number of commits (with additional map-update costs). Neither greedy reuse nor checkpoint caching eliminates this worst case or guarantees an optimal number of chains.